Luiz Henrique Gariglio dos Santos

Optimisation Par Colonie De Fourmi D'un Chemin Sur Un Paysage Fractal

PALAISEAU
2026

TABLE DES MATIÈRES

1 Colonie de Fourmis **2**
2 MESURE DU TEMPS PASSÉ PAR ITÉRATION **2**
3 VECTORISATION ET OPENMP **2**
4 Parallélisation **3**
5 Conclusion **4**

1 Colonie de Fourmis

Le projet cherche a étudier comme différentes techniques peuvent être utilisées pour l'amélioration d'un algorithme. L'algorithme utilisé comme base, fourni par le professeur, implémente une simulation de colonies de fourmis sur un terrain fractal visualisée en temps réel. L'objectif de cet algorithme est d'observer comment un grand nombre de fourmis, suivant des règles locales simples, parvient à découvrir un chemin efficace entre un nid et une source de nourriture.

Lien vers le github du projet: Colonie des Fourmis

2 MESURE DU TEMPS PASSÉ PAR ITÉRATION

Dans ce premier moment, nous allons mesurer le temps d'exécution pour chaque itération de l'algorithme et le temps pour montrer le *display*. Pour le faire, j'ai utilisé la librairie *chrono* et j'ai défini un nombre maximale d'itérations. De cette façon, le temps a été mesuré comme la valeur moyenne du temps pris dans chaque itération. Les résultats sont présentées dans la figure 1.

```
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Initial$ ./ant_simu.exe
La première nourriture est arrivée au nid a l'iteration 2934
Iter 5000 advance_time = 5.13558 ms, render = 11.4857 ms
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Initial$ ./ant_simu.exe
La première nourriture est arrivée au nid a l'iteration 2680
Iter 5000 advance_time = 5.07078 ms, render = 11.5506 ms
```

Figure 1 – Temps de calcul du code initial

3 VECTORISATION ET OPENMP

Une modification possible est de vectoriser les données. La vectorisation transforme les traitements individuels (par fourmi) en opérations sur tableaux (coordonnées, états, graines des fourmis). Ce qui rendre des avantages comme un gain de performance, car les processeurs modernes exécutent plusieurs opérations simultanément sur des vecteurs. L'accès contigu aux données optimise aussi la localité et réduit les *overheads* de cache. Finalement, les tableaux vectorisés facilitent l'usage d'OpenMP pour paralléliser les boucles, améliorant l'accélération en fonction du nombre de threads.

Pour implémenter la vectorisation, la classe *ant* a été remplacé par quatre vecteurs différents (*pos_x, pos_y, seeds et loaded*). La classe *renderer* a été modifié pour recevoir des vecteurs au lieu d'une structure et le méthode *advance* de la classe *ants* a été inclu dans la foction *advance_time*.

```
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Vectorisation$ ./ant_simu.exe
La première nourriture est arrivée au nid a l'iteration 2585
Iter 5000 advance_time = 4.88948 ms, render = 11.7373 ms
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Vectorisation$ ./ant_simu.exe
La première nourriture est arrivée au nid a l'iteration 2585
Iter 5000 advance_time = 4.94279 ms, render = 11.6899 ms
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Vectorisation$ ./ant_simu.exe
La première nourriture est arrivée au nid a l'iteration 2585
Iter 5000 advance_time = 5.09892 ms, render = 11.533 ms
```

Figure 2 – Temps de calcul du code vectorisé

Le résultats de la vectorisation peuvent être visualisés dans la figure 2. Et après voir les temps d'itération nous constatons que la vectorisation par elle même n'est pas une garantie d'amélioration. Et les valeurs n'ont peut-être pas changé pour différents raisons comme des dépendances mémoires.

Pour cette raison le *pragma omp parallel for* a été utilisé pour garantir la parallélisation d'un boucle dans la fonction *advance_time*. Les résultats sont montrées dans l'image 3.

```
● (base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Vectorisation$ OMP_NUM_THREADS=2 ./ant_simu.exe
  La première nourriture est arrivée au nid a l'iteration 2585
  Iter 5000 advance_time = 2.62823 ms, render = 13.9978 ms
● (base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Vectorisation$ OMP_NUM_THREADS=4 ./ant_simu.exe
  La première nourriture est arrivée au nid a l'iteration 2585
  Iter 5000 advance_time = 1.97532 ms, render = 14.6493 ms
● (base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Vectorisation$ OMP_NUM_THREADS=8 ./ant_simu.exe
  La première nourriture est arrivée au nid a l'iteration 2585
  Iter 5000 advance_time = 1.68039 ms, render = 14.9449 ms
```

Figure 3 – Temps de calcul du code parallélisé

Code	Temps [ms]	Speed-up[%]
Advance_time (vect)	4.97	1.03
Render (vect)	11.65	0.99
Advance_time (thread = 2)	2.63	1.94
Render (thread = 2)	13.99	0.82
Advance_time (thread = 4)	1.97	2.59
Render (thread = 4)	14.65	0.79
Advance_time (thread = 8)	1.68	3.04
Render (thread = 8)	14.94	0.77

Table 1 – Speed-Up vectorisation

4 Parallélisation

Une autre approche d'amélioration est avec la parallélisation avec *MPI* dont chaque processus gére une partie des fourmis. Dans cette implémentation MPI, chaque processus possède une copie complète de l'environnement fractal mais ne contrôle qu'une sous-colonie de fourmis. Ça résulte das un parallélisme massif des fourmis une fois que diviser m fourmis entre P processus permet d'explorer simultanément P fois plus de chemins. Cette approche a aussi une bonne équilibrage de charge car chaque processus traite exactement m/P fourmis, indépendamment de leur position géographique sur la grille.

Pour implémenter la parallélisation j'ai utilisé la librairie *mpi.h*. Le processus de *rank* zero est le seule responsable pour l'affichage de la fenêtre. Chaque processus contrôle touts les paramètres de m/P fourmis localement. Après les données sont aussi rajoutés dans le processus zero. Pour le pheromone, une nouvelle fonction a été crée comme méthode de la classe pour le synchroniser. Ce méthode reçoit le envoie les données de pheromones vers le processus zero, calcule la moyenne et met a jour les données de chaque processus. Les données de temps sont présentés dans la figure 4 et le *Speed-up* en relation au temps initial dans la table 2.

```
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Parallelisation$ mpirun -np 1 ./ant_simu.exe
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
La première nourriture est arrivée au nid a l'iteration 2585
Iter 5000rank 0 advance_time = 4.66318 ms, render = 10.3701 ms
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Parallelisation$ mpirun -np 2 ./ant_simu.exe
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
La première nourriture est arrivée au nid a l'iteration 2583
Iter 5000rank 1 advance_time = 3.50732 ms
Iter 5000rank 0 advance_time = 3.31047 ms, render = 5.92312 ms
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Parallelisation$ mpirun -np 4 ./ant_simu.exe
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
La première nourriture est arrivée au nid a l'iteration 2340
Iter 5000rank 1 advance_time = 3.08874 ms
Iter 5000rank 2 advance_time = 2.82387 ms
Iter 5000rank 3 advance_time = 3.18137 ms
Iter 5000rank 0 advance_time = 2.7189 ms, render = 5.57041 ms
(base) luiz@luiz:~/Documentos/Ensta/Systemes-paralleles/Parallelisation$ mpirun -np 8 ./ant_simu.exe
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
La première nourriture est arrivée au nid a l'iteration 2031
Iter 5000rank 7 advance_time = 5.18199 ms
Iter 5000rank 1 advance_time = 5.51516 ms
Iter 5000rank 2 advance_time = 5.88741 ms
Iter 5000rank 3 advance_time = 5.60504 ms
Iter 5000rank 4 advance_time = 5.26723 ms
Iter 5000rank 5 advance_time = 5.59435 ms
Iter 5000rank 6 advance_time = 5.42154 ms
Iter 5000rank 0 advance_time = 2.81533 ms, render = 8.37196 ms
```

Figure 4 – Temps de calcul du code parallélisé

Code	Temps [ms]	Speed-up[%]
Advance_time (thread = 2)	3.5	1.46
Render (thread = 2)	5.92	1.94
Advance_time (thread = 4)	3.1	1.64
Render (thread = 4)	5.57	2.07
Advance_time (thread = 8)	5.8	0.88
Render (thread = 8)	8.37	1.38

Table 2 – Speed-Up parallélisation

5 Conclusion

Ce projet a démontré l'efficacité des techniques d'optimisation pour accélérer l'algorithme d'optimisation par colonie de fourmis sur un paysage fractal. La vectorisation, bien que limitée a préparé le terrain pour la parallélisation OpenMP, qui a délivré un *speed-up* plus important. Même que la vectorisation avec le OpenMP a obtenu une diminution du temps pour *l'advance_time*, la fonction *render* a subi une augmentation du temps d'itération. À cause, probablement, d'une concurrence pour le processeur. Et si nous considérons le temps total, le gain est presque nul.

La parallélisation, au contraire, était vraiment efficace décroissant le temps pour *l'advance_time*, et pour le *render*. Même que la diminution pour *l'advance_time* n'a pas été comme dans la vectorisation, elle était encore bonne. Et la diminution pour le *render* a été plus importante une fois qu'elle prenait plus de temps en comparaison avec l'autre fonction.

De toute façon, est important souligner qu'il y a un limite dans le nombre de *threads* utilisée pour avoir de bonnes résultats. Dans la table 2 nous pouvons voire que le résultat avec quatre *threads* a été meilleure que celui avec huit.